



CSE 711 - Compilers

Lecture 3 (Part 1)

Syntax Analysis

Ms. Samia Muntaha

Lecturer,

Department of Computer Science and Engineering
University of Chittagong

Outline

- ▶ Types of Grammars
- ▶ Context Free Grammar
- ▶ Leftmost and Rightmost Derivation
- ▶ Ambiguity
- ▶ Left Recursion
- ▶ Elimination of left recursion
- ▶ Left Factoring

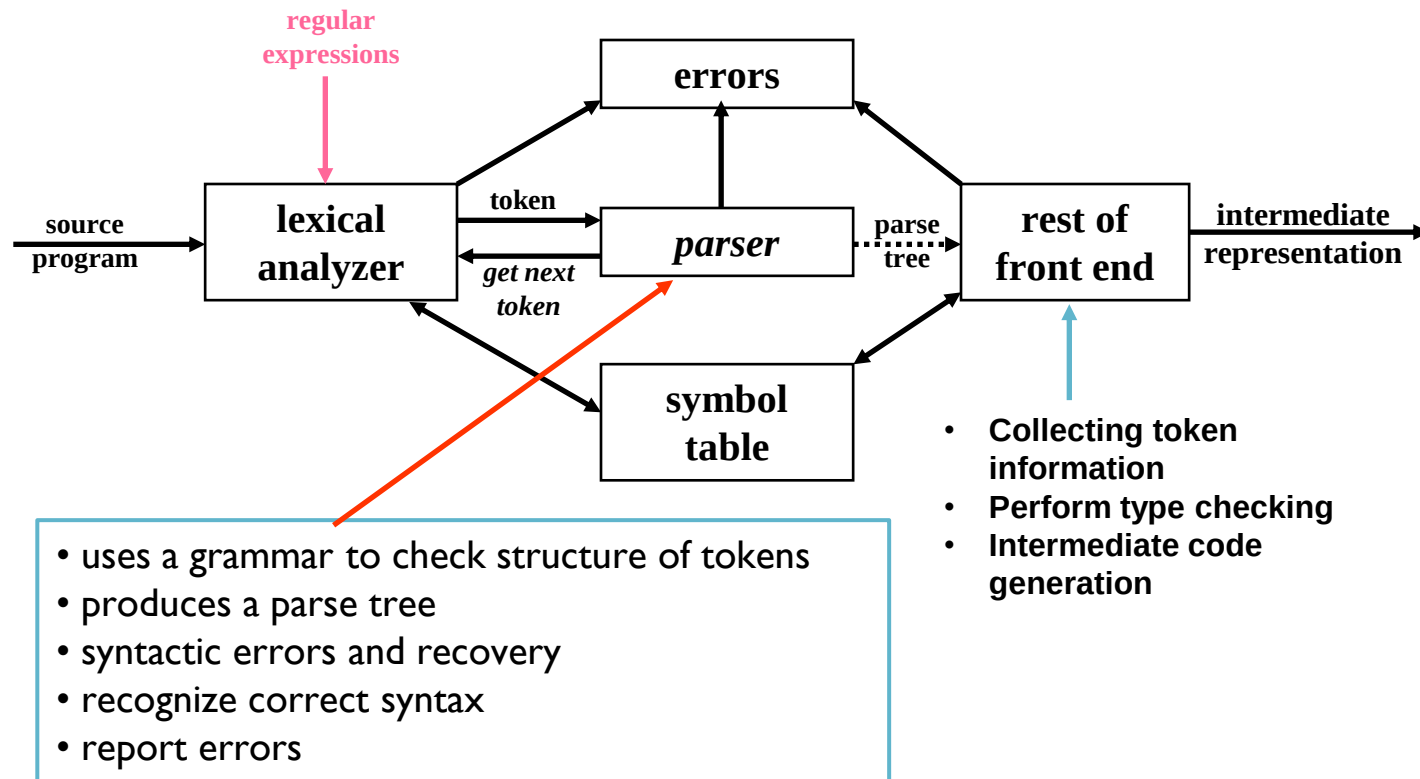
Language and Grammars

- Every (programming) language has precise rules
 - In English:
 - Subject Verb Object
 - In C
 - programs are made of functions
 - » Functions are made of statements etc.

Parsing

- A.K.A. Syntax Analysis
 - Recognize sentences in a language.
 - Discover the structure of a document/program.
 - Construct (implicitly or explicitly) a tree (called as a parse tree) to represent the structure.
 - The above tree is used later to guide translation.

Parsing During Compilation



Errors in Programs

- **Lexical**

if $x < 1$ then $y = 5$;

“Typos”

- **Syntactic**

if $((x < 1) \ \& \ (y > 5))$...

{ ... { ... _ ... } }

- **Semantic**

if $(x + 5)$ then ...

Type Errors

Undefined IDs, etc.

- **Logical Errors**

if $(i < 9)$ then ...

Should be $\leq$ not $<$

Bugs

Compiler cannot detect Logical Errors

Error Detection

- Much responsibility on Parser
 - Many errors are syntactic in nature
 - Precision/ efficiency of modern parsing method
 - Detect the error as soon as possible
- Challenges for error handler in Parser
 - Report error clearly and accurately
 - Recover from error and continue..
 - Should be efficient in processing
- Good news is
 - Simple mechanism can catch most common errors
- Errors don't occur that frequently!!
 - 60% programs are syntactically and semantically correct
 - 80% erroneous statements have only 1 error, 13% have 2
 - Most error are trivial : 90% single token error
 - 60% punctuation, 20% operator, 15% keyword, 5% other error

Adequate Error Reporting is Not a Trivial Task

- Difficult to generate clear and accurate error messages.

Example

```
function foo () {  
  ...  
  if (...) {  
    ...  
  } else {  
    ...  
    ...  
  }  
  ...  
}  
<eof>
```

Missing } here

Not detected until here

Example

```
int myVarr;  
...  
x = myVar;  
...
```

Misspelled ID here

Not detected until here

ERROR RECOVERY

- After first error recovered
 - Compiler must go on!
 - Restore to some state and process the rest of the input
- **Error-Correcting Compilers**
 - Issue an error message
 - Fix the problem
 - Produce an executable

Example

```
Error on line 23: "myVarr" undefined.  
"myVar" was used.
```

May not be a good Idea!!

- Guessing the programmers intention is not easy!

ERROR RECOVERY MAY TRIGGER MORE ERRORS!

- Inadequate recovery may introduce more errors
 - Those were not programmers errors

- Example:

```
int myVar flag ;
```

```
...
```

```
x := flag;
```

```
...
```

```
...
```

```
while (flag==0)
```

```
...
```

Declaration of flag is discarded

Variable flag is undefined

Variable falg is undefined

Too many Error message may be obscuring

- May bury the real message
- Remedy:
 - allow 1 message per token or per statement
 - Quit after a maximum (e.g. 100) number of errors

ERROR RECOVERY APPROACHES: PANIC MODE

- Discard tokens until we see a “synchronizing” token.

Example

Skip to next occurrence of
`} end ;`
Resume by parsing the next statement

- The key...
 - Good set of synchronizing tokens
 - Knowing what to do then
- Advantage
 - Simple to implement
 - Does not go into infinite loop
 - Commonly used
- Disadvantage
 - May skip over large sections of source with some errors

ERROR RECOVERY APPROACHES:

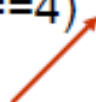
PHRASE-LEVEL RECOVERY

- Compiler corrects the program
by deleting or inserting tokens
...so it can proceed to parse from where it was.

Example

while (x==4) y:= a + b

Insert **do** to fix the statement



- The key...
Don't get into an infinite loop
...constantly inserting tokens and never scanning the actual source
- Generally used for **error-repairing** compilers
 - Difficulty: Point of error detection might be much later the point of error occurrence

ERROR RECOVERY APPROACHES: ERROR PRODUCTIONS

- Augment the CFG with “Error Productions”
- Now the CFG accepts anything!
- If “error productions” are used...

 Their actions:

 { **print (“Error...”)** }

- Used with...
 - LR (Bottom-up) parsing
 - Parser Generators

ERROR RECOVERY APPROACHES:

GLOBAL CORRECTION

- Theoretical Approach
- Find the minimum change to the source to yield a valid program
 - Insert tokens, delete tokens, swap adjacent tokens
- Global Correction Algorithm
 - Input:** grammatically incorrect input string x ; grammar G
 - Output:** grammatically correct string y
 - Algorithm:** converts $x \rightarrow y$ using minimum number changes (insertion, deletion etc.)
- Impractical algorithms - too time consuming

Parsers

- We categorize the parsers into two groups:

1. Top-Down Parser

- the parse tree is created top to bottom, starting from the root.

2. Bottom-Up Parser

- the parse is created bottom to top; starting from the leaves

- Both top-down and bottom-up parsers scan the input from left to right (one symbol at a time).
- Efficient top-down and bottom-up parsers can be implemented only for sub-classes of context-free grammars.
 - LL for top-down parsing
 - LR for bottom-up parsing

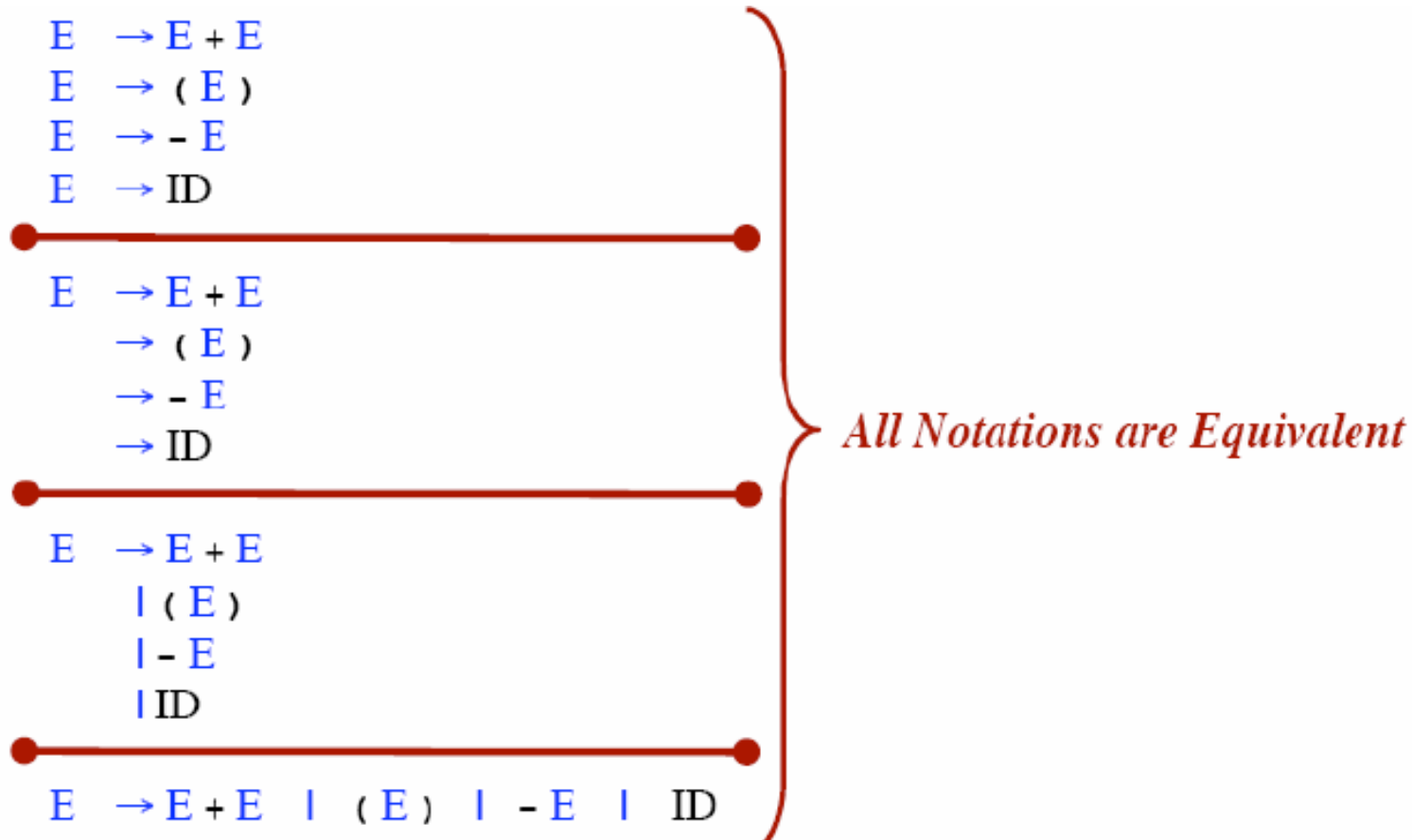
CONTEXT FREE GRAMMARS (CFG)

- ▶ A context-free grammar has four components: $G = (V, \Sigma, P, S)$
- ✓ A set of **non-terminals** (V). Non-terminals are syntactic variables that denote sets of strings. The non-terminals define sets of strings that help define the language generated by the grammar.
- ✓ A set of tokens, known as **terminal symbols** (Σ). Terminals are the basic symbols from which strings are formed.
- ✓ A set of **productions** (P). The productions of a grammar specify the manner in which the terminals and non-terminals can be combined to form strings. Each production consists of a **non-terminal** called the left side of the production, an arrow, and a sequence of tokens and/or **on-terminals**, called the right side of the production.
- ✓ One of the non-terminals is designated as the **start symbol** (S); from where the production begins.

Example of CFG:

- ▶ $G = (V, \Sigma, P, S)$ Where:
- ▶ $V = \{ Q, Z, N \}$
- ▶ $\Sigma = \{ 0, 1 \}$
- ▶ $P = \{ Q \rightarrow Z \mid Q \rightarrow N \mid Q \rightarrow \varepsilon \mid Z \rightarrow 0Q0 \mid N \rightarrow 1Q1 \}$
- ▶ $S = \{ Q \}$
- This grammar describes palindrome language, such as: 1001, 11100111, 00100, 1010101, 11111, etc.

RULE ALTERNATIVE NOTATIONS



NOTATIONAL CONVENTIONS

Terminals

a b c ...

Nonterminals

A B C ...

S

Expr

Grammar Symbols (Terminals or Nonterminals)

X Y Z U V W ...

Strings of Symbols

α β γ ...

A sequence of zero
Or more terminals
And nonterminals

Strings of Terminals

x y z u v w ...

Including ϵ

Examples

$A \rightarrow \alpha B$

A rule whose righthand side ends with a nonterminal

$A \rightarrow x \alpha$

A rule whose righthand side begins with a string of terminals (call it "x")

DERIVATIONS

- ❑ A derivation is basically a sequence of production rules, in order to get the input string. During parsing, we take two decisions for some sentential form of input:
- ❑ Deciding the non-terminal which is to be replaced.
- ❑ Deciding the production rule, by which, the non-terminal will be replaced.
- ❑ To decide which non-terminal to be replaced with production rule, we can have two options.

DERIVATIONS

1. $E \rightarrow E + E$
2. $\rightarrow E * E$
3. $\rightarrow (E)$
4. $\rightarrow - E$
5. $\rightarrow ID$

A “Derivation” of “(id*id)”

$$E \Rightarrow (E) \Rightarrow (E * E) \Rightarrow (\underline{id} * E) \Rightarrow (\underline{id} * \underline{id})$$


“Sentential Forms”

A sequence of terminals and nonterminals in a derivation

(id * E)

DERIVATIONS

If $A \rightarrow \beta$ is a rule, then we can write

$$\underbrace{\alpha A \gamma}_{\uparrow} \Rightarrow \alpha \beta \gamma$$

*Any sentential form containing a nonterminal (call it A)
... such that A matches the nonterminal in some rule.*

Derives in zero-or-more steps $\Rightarrow^*$

$$E \Rightarrow^* (\underline{\text{id}} * \underline{\text{id}})$$

If $\alpha \Rightarrow^* \beta$ and $\beta \Rightarrow \gamma$, then $\alpha \Rightarrow^* \gamma$

Derives in one-or-more steps $\Rightarrow^+$

CFG Terminology

Given

G A grammar
S The Start Symbol

Define

$L(G)$ The language generated
 $L(G) = \{ w \mid S \Rightarrow^+ w \}$

“Equivalence” of CFG’s

If two CFG’s generate the same language, we say they are “**equivalent**.”

$$G_1 \approx G_2 \text{ whenever } L(G_1) = L(G_2)$$

In making a derivation...

Choose which nonterminal to expand

Choose which rule to apply

LEFTMOST DERIVATION

In a derivation... always expand the leftmost nonterminal.

E
 $\Rightarrow E + E$
 $\Rightarrow (E) + E$
 $\Rightarrow (E * E) + E$
 $\Rightarrow (\underline{id} * E) + E$
 $\Rightarrow (\underline{id} * \underline{id}) + E$
 $\Rightarrow (\underline{id} * \underline{id}) + \underline{id}$

- | | |
|----|-----------------------|
| 1. | $E \rightarrow E + E$ |
| 2. | $\rightarrow E * E$ |
| 3. | $\rightarrow (E)$ |
| 4. | $\rightarrow - E$ |
| 5. | $\rightarrow ID$ |

Let $\Rightarrow_{LM}$ denote a step in a leftmost derivation ($\Rightarrow_{LM}^*$ means zero-or-more steps)

At each step in a leftmost derivation, we have

$wA\gamma \Rightarrow_{LM} w\beta\gamma$ where $A \rightarrow \beta$ is a rule

(Recall that w is a string of terminals.)

Each sentential form in a leftmost derivation is called a “**left-sentential form.**”

If $S \Rightarrow_{LM}^* \alpha$ then we say α is a “**left-sentential form.**”

RIGHTMOST DERIVATION

In a derivation... always expand the rightmost nonterminal.

E
 $\Rightarrow E + E$
 $\Rightarrow E + \underline{id}$
 $\Rightarrow (E) + \underline{id}$
 $\Rightarrow (E * E) + \underline{id}$
 $\Rightarrow (E * \underline{id}) + \underline{id}$
 $\Rightarrow (\underline{id} * \underline{id}) + \underline{id}$

- | | |
|----|-----------------------|
| 1. | $E \rightarrow E + E$ |
| 2. | $\rightarrow E * E$ |
| 3. | $\rightarrow (E)$ |
| 4. | $\rightarrow - E$ |
| 5. | $\rightarrow ID$ |

Let $\Rightarrow_{\text{RM}}$ denote a step in a rightmost derivation ($\Rightarrow_{\text{RM}}^*$ means zero-or-more steps)

At each step in a rightmost derivation, we have

$$\alpha A w \Rightarrow_{\text{RM}} \alpha \beta w \quad \text{where } A \rightarrow \beta \text{ is a rule}$$

(Recall that w is a string of terminals.)

Each sentential form in a rightmost derivation is called a “**right-sentential form**.”

If $S \Rightarrow_{\text{RM}}^* \alpha$ then we say α is a “**right-sentential form**.”

PARSE TREE

- A **parse tree** is a graphical representation of a derivation sequence of a sentential form.
- Tree nodes represent symbols of the grammar (nonterminals or terminals) and tree edges represent derivation steps.
- Inner nodes of a parse tree are non-terminal symbols.
- The leaves of a parse tree are terminal symbols.

PARSE TREE

Two choices at each step in a derivation...

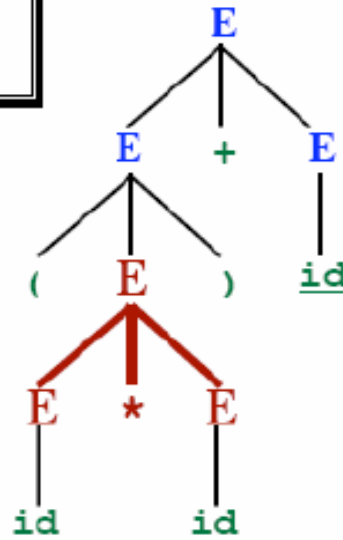
- Which non-terminal to expand
- Which rule to use in replacing it

The parse tree remembers only this

Leftmost Derivation:

E
 $\Rightarrow E + E$
 $\Rightarrow (E) + E$
 $\Rightarrow (E * E) + E$
 $\Rightarrow (\underline{id} * E) + E$
 $\Rightarrow (\underline{id} * \underline{id}) + E$
 $\Rightarrow (\underline{id} * \underline{id}) + \underline{id}$

1. $E \rightarrow E + E$
2. $\rightarrow E * E$
3. $\rightarrow (E)$
4. $\rightarrow - E$
5. $\rightarrow ID$



PARSE TREE

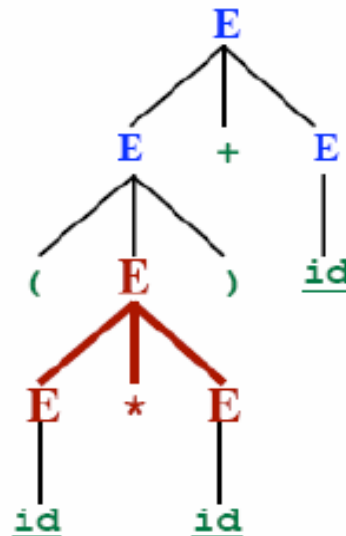
Two choices at each step in a derivation...

- Which non-terminal to expand
- Which rule to use in replacing it

The parse tree remembers only this

Rightmost Derivation:

E
 $\Rightarrow E + E$
 $\Rightarrow E + \underline{id}$
 $\Rightarrow (E) + \underline{id}$
 $\Rightarrow (E * E) + \underline{id}$
 $\Rightarrow (E * \underline{id}) + \underline{id}$
 $\Rightarrow (\underline{id} * \underline{id}) + \underline{id}$



1. $E \rightarrow E + E$
2. $\rightarrow E * E$
3. $\rightarrow (E)$
4. $\rightarrow - E$
5. $\rightarrow ID$

PARSE TREE

Two choices at each step in a derivation...

- Which non-terminal to expand
- Which rule to use in replacing it

The parse tree remembers only this

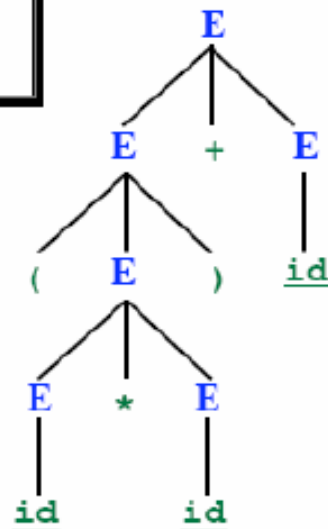
Leftmost Derivation:

E
 $\Rightarrow E + E$
 $\Rightarrow (E) + E$
 $\Rightarrow (E * E) + E$
 $\Rightarrow (\underline{id} * E) + E$
 $\Rightarrow (\underline{id} * \underline{id}) + E$
 $\Rightarrow (\underline{id} * \underline{id}) + \underline{id}$

Rightmost Derivation:

E
 $\Rightarrow E + E$
 $\Rightarrow E + \underline{id}$
 $\Rightarrow (E) + \underline{id}$
 $\Rightarrow (E * E) + \underline{id}$
 $\Rightarrow (E * \underline{id}) + \underline{id}$
 $\Rightarrow (\underline{id} * \underline{id}) + \underline{id}$

1. $E \rightarrow E + E$
2. $\rightarrow E * E$
3. $\rightarrow (E)$
4. $\rightarrow - E$
5. $\rightarrow ID$



PARSE TREE

Given a leftmost derivation, we can build a parse tree.

Given a rightmost derivation, we can build a parse tree.

Leftmost Derivation of

(id*id)+id

Rightmost Derivation of

(id*id)+id



Same Parse Tree

Every parse tree corresponds to...

- A single, unique leftmost derivation
- A single, unique rightmost derivation

AMBIGUOUS GRAMMAR

Ambiguity:

However, one input string may have several parse trees!!!

Therefore:

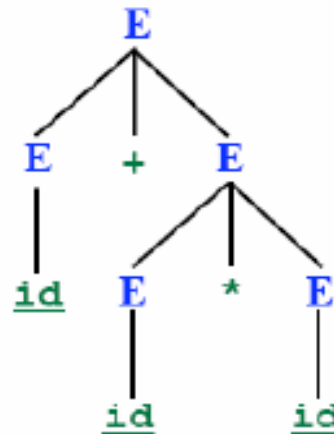
- Several leftmost derivations
- Several rightmost derivations

A grammar that produces more than one parse tree for any input sentence is said to be an **ambiguous** grammar.

AMBIGUOUS GRAMMAR

Leftmost Derivation #1

E
 $\Rightarrow E + E$
 $\Rightarrow \underline{id} + E$
 $\Rightarrow \underline{id} + E * E$
 $\Rightarrow \underline{id} + \underline{id} * E$
 $\Rightarrow \underline{id} + \underline{id} * \underline{id}$

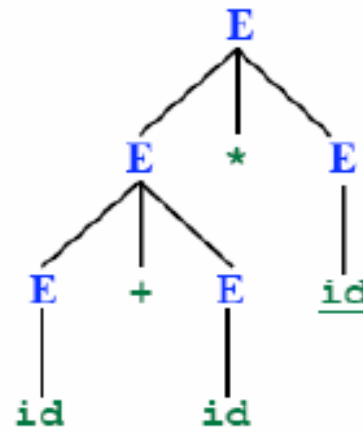


1. $E \rightarrow E + E$
2. $\rightarrow E * E$
3. $\rightarrow (E)$
4. $\rightarrow - E$
5. $\rightarrow ID$

Input: $id + id * id$

Leftmost Derivation #2

E
 $\Rightarrow E * E$
 $\Rightarrow E + E * E$
 $\Rightarrow \underline{id} + E * E$
 $\Rightarrow \underline{id} + \underline{id} * E$
 $\Rightarrow \underline{id} + \underline{id} * \underline{id}$



AMBIGUOUS GRAMMAR

$E ::= E + E$

$E ::= E - E$

$E ::= E * E$

$E ::= E / E$

$E ::= (E)$

$E ::= \text{num}$

$E ::= \text{id}$

➤ *Is this an ambiguous grammar?*

➤ **Example:**

- Find a derivation for the expression: **4 / 2 + 2**

AMBIGUOUS GRAMMAR

- Why are ambiguous grammars problematic?

$$(4 / 2) + 2 = 4 \quad \text{or} \quad 4 / (2 + 2) = 1$$

- It is often possible to transform an ambiguous grammar into an equivalent unambiguous grammar.
- In our grammar,
 - * has higher precedence than +
 - each operator associates to the left

AMBIGUOUS GRAMMAR

$E ::= E + E$

$E ::= E - E$

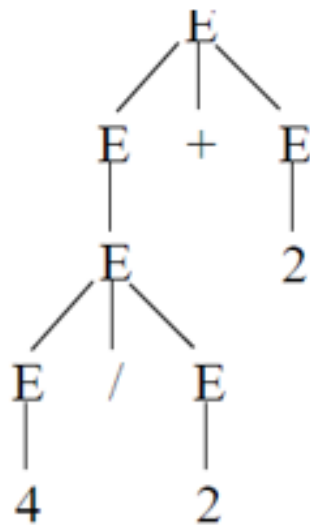
$E ::= E * E$

$E ::= E / E$

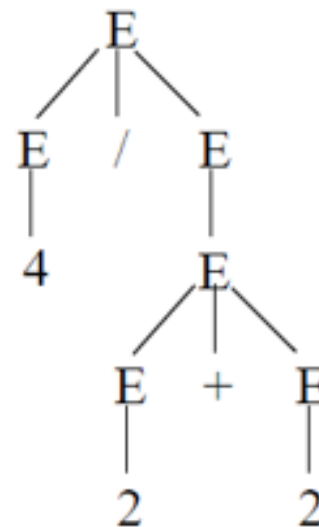
$E ::= (E)$

$E ::= \text{num}$

$E ::= \text{id}$



4 / 2 + 2



AMBIGUOUS GRAMMAR

- For the most parsers, the grammar must be unambiguous.
- unambiguous grammar
 - ➔ unique selection of the parse tree for a sentence
- *We should eliminate the ambiguity in the grammar during the design phase of the compiler.*
- An **unambiguous** grammar should be written to eliminate the ambiguity.
- We have to prefer one of the parse trees of a sentence (generated by an ambiguous grammar) to disambiguate that grammar to restrict to this choice.

AMBIGUOUS GRAMMAR

➤ What about this grammar?

$E ::= E + E$

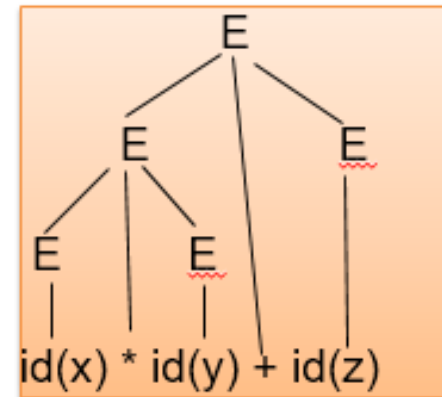
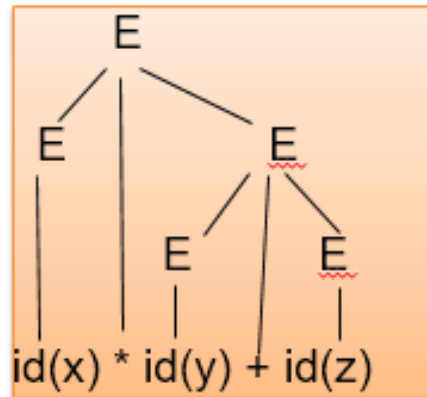
$| E - E$

$| E * E$

$| E / E$

$| \text{num}$

$| \text{id}$



- Operators **$+$** **$-$** **$*$** **$/$** have the same precedence!
- It is **ambiguous**: has more than one parse tree for the same input sequence (depending which derivations are applied each time)

UNAMBIGUOUS GRAMMAR

$E ::= E + E$

$E ::= E - E$

$E ::= E * E$

$E ::= E / E$

$E ::= (E)$

$E ::= \text{num}$

$E ::= \text{id}$

Ambiguous to
unambiguous
grammar

$E ::= E + T$

$E ::= E - T$

$E ::= T$

$T ::= T * F$

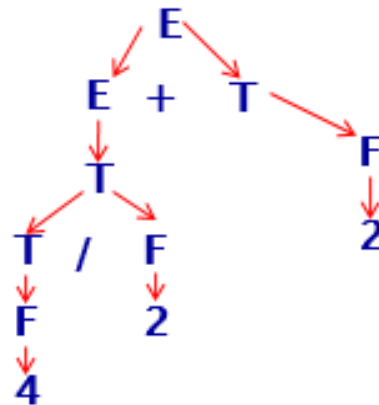
$T ::= T / F$

$T ::= F$

$F ::= \text{id}$

$F ::= \text{num}$

$F ::= (E)$



PREDICTIVE PARSING

- The goal is to construct a top-down parser that never backtracks
- Always leftmost derivations
- We must transform a grammar in two ways:
 - eliminate left recursion
 - perform left factoring
- These rules eliminate most common causes for backtracking although they do not guarantee a completely backtrack-free parsing

LEFT RECURSION: INFINITE LOOPING PROBLEM

A grammar is left-recursive if it has a non-terminal A, such that there is a derivation :

$$A \xRightarrow{+} A\alpha, \text{ for some } \alpha.$$

Top-Down parsing can't reconcile this type of grammar, **since it could consistently make choice which wouldn't allow termination.**

$$A \Rightarrow A\alpha \Rightarrow A\alpha\alpha \Rightarrow A\alpha\alpha\alpha \dots \text{etc. } A \rightarrow A\alpha \mid \beta$$

So we have to convert our left-recursive grammar into an equivalent grammar which is not left-recursive.

The left-recursion may appear in a single step of the derivation (*immediate left-recursion*), or may appear in more than one step of the derivation.

IMMEDIATE LEFT RECURSION

➤ $A \rightarrow A \alpha \mid \beta$ where β does not start with A

$\Downarrow$ eliminate immediate left recursion

$A \rightarrow \beta A'$ where A' is a new nonterminal

$A' \rightarrow \alpha A' \mid \varepsilon$ an equivalent grammar

More General (but still immediate):

$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid A\alpha_3 \mid \dots \mid \beta_1 \mid \beta_2 \mid \beta_3 \mid \dots$

Transform into:

$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \beta_3 A' \mid \dots$

$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \alpha_3 A' \mid \dots \mid \varepsilon$

IMMEDIATE LEFT RECURSION ELIMINATION: EXAMPLE

Our Example :

$$\begin{array}{lcl} E \rightarrow E + T \mid T & \longrightarrow & \left\{ \begin{array}{l} E \rightarrow TE' \\ E' \rightarrow + TE' \mid \epsilon \end{array} \right. \\ T \rightarrow T * F \mid F & \longrightarrow & \left\{ \begin{array}{l} T \rightarrow FT' \\ T' \rightarrow * FT' \mid \epsilon \end{array} \right. \\ F \rightarrow (E) \mid id & \longrightarrow & F \rightarrow (E) \mid id \end{array}$$

LEFT RECURSION IN MORE THAN ONE STEP

- A grammar cannot be immediately left-recursive, but it still can be left-recursive.
- By just eliminating the immediate left-recursion, we may not get a grammar which is not left-recursive.

Example:

$S \rightarrow A\underline{f} \mid \underline{b}$

$A \rightarrow A\underline{c} \mid S\underline{d} \mid \underline{e}$

Is A left recursive? Yes.

Is S left recursive? Yes, but not immediate left recursion. $S \Rightarrow A\underline{f} \Rightarrow S\underline{d}\underline{f}$

LEFT RECURSION IN MORE THAN ONE STEP: ELIMINATION

Approach:

Look at the rules for S only (ignoring other rules)... No left recursion.
Look at the rules for A ...

Do any of A 's rules start with S ? Yes.

$$A \rightarrow S\underline{d}$$

Get rid of the S . Substitute in the righthand sides of S .

$$A \rightarrow A\underline{f}\underline{d} \mid \underline{b}\underline{d}$$

The modified grammar:

$$S \rightarrow A\underline{f} \mid \underline{b}$$

$$A \rightarrow A\underline{c} \mid A\underline{f}\underline{d} \mid \underline{b}\underline{d} \mid \underline{e}$$

Now eliminate immediate left recursion involving A .

$$S \rightarrow A\underline{f} \mid \underline{b}$$

$$A \rightarrow \underline{b}\underline{d}A' \mid \underline{e}A'$$

$$A' \rightarrow \underline{c}A' \mid \underline{f}\underline{d}A' \mid \underline{\epsilon}$$

LEFT RECURSION IN MORE THAN ONE STEP: ELIMINATION

The Original Grammar:

$$S \rightarrow A\underline{f} \mid \underline{b}$$
$$A \rightarrow A\underline{c} \mid S\underline{d} \mid B\underline{e}$$
$$B \rightarrow A\underline{g} \mid S\underline{h} \mid \underline{k}$$

So Far:

$$S \rightarrow A\underline{f} \mid \underline{b}$$
$$A \rightarrow \underline{b}dA' \mid B\underline{e}A'$$
$$A' \rightarrow \underline{c}A' \mid \underline{f}dA' \mid \epsilon$$

LEFT RECURSION IN MORE THAN ONE STEP: ELIMINATION

The Original Grammar:

$S \rightarrow A\underline{f} \mid \underline{b}$

$A \rightarrow A\underline{c} \mid S\underline{d} \mid B\underline{e}$

$B \rightarrow A\underline{g} \mid S\underline{h} \mid \underline{k}$

So Far:

$S \rightarrow A\underline{f} \mid \underline{b}$

$A \rightarrow \underline{b}dA' \mid B\underline{e}A'$

$A' \rightarrow \underline{c}A' \mid \underline{f}dA' \mid \epsilon$

$B \rightarrow A\underline{g} \mid S\underline{h} \mid \underline{k}$

Look at the B rules next;
Does any righthand side
start with “S”?

LEFT RECURSION IN MORE THAN ONE STEP: ELIMINATION

The Original Grammar:

$S \rightarrow A\underline{f} \mid \underline{b}$

$A \rightarrow A\underline{c} \mid S\underline{d} \mid B\underline{e}$

$B \rightarrow A\underline{g} \mid S\underline{h} \mid \underline{k}$

So Far:

$S \rightarrow A\underline{f} \mid \underline{b}$

$A \rightarrow \underline{b}dA' \mid B\underline{e}A'$

$A' \rightarrow \underline{c}A' \mid \underline{f}dA' \mid \epsilon$

$B \rightarrow A\underline{g} \mid A\underline{f}h \mid \underline{b}h \mid \underline{k}$

Substitute, using the rules for “S”

$A\underline{f}... \mid \underline{b}...$

LEFT RECURSION IN MORE THAN ONE STEP: ELIMINATION

The Original Grammar:

$S \rightarrow A\underline{f} \mid \underline{b}$

$A \rightarrow A\underline{c} \mid S\underline{d} \mid B\underline{e}$

$B \rightarrow A\underline{g} \mid S\underline{h} \mid \underline{k}$

So Far:

$S \rightarrow A\underline{f} \mid \underline{b}$

$A \rightarrow \underline{b}dA' \mid B\underline{e}A'$

$A' \rightarrow \underline{c}A' \mid \underline{f}dA' \mid \epsilon$

$B \rightarrow \underline{A}g \mid \underline{A}f\underline{h} \mid \underline{b}h \mid \underline{k}$

Does any righthand side
start with “A”?

LEFT RECURSION IN MORE THAN ONE STEP: ELIMINATION

The Original Grammar:

$S \rightarrow A\underline{f} \mid \underline{b}$

$A \rightarrow A\underline{c} \mid S\underline{d} \mid B\underline{e}$

$B \rightarrow A\underline{g} \mid S\underline{h} \mid \underline{k}$

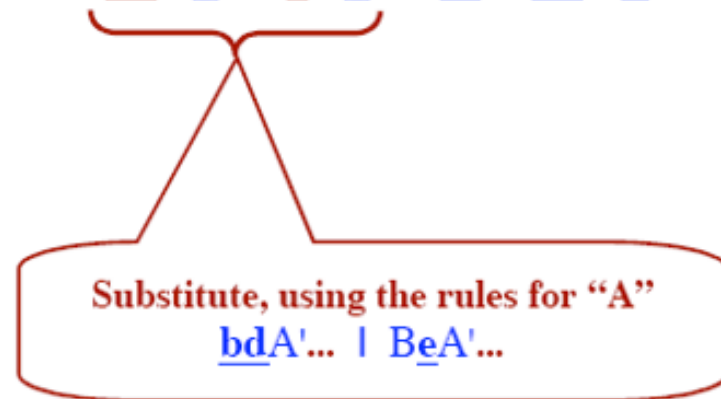
So Far:

$S \rightarrow A\underline{f} \mid \underline{b}$

$A \rightarrow \underline{b}dA' \mid B\underline{e}A'$

$A' \rightarrow \underline{c}A' \mid \underline{f}dA' \mid \epsilon$

$B \rightarrow \underline{b}dA'g \mid B\underline{e}A'g \mid A\underline{f}h \mid \underline{b}h \mid \underline{k}$



LEFT RECURSION IN MORE THAN ONE STEP: ELIMINATION

The Original Grammar:

$S \rightarrow A\bar{f} \mid \bar{b}$

$A \rightarrow A\bar{c} \mid S\bar{d} \mid B\bar{e}$

$B \rightarrow A\bar{g} \mid S\bar{h} \mid \bar{k}$

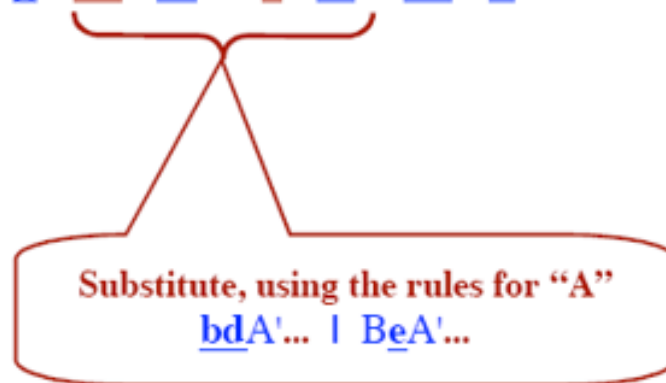
So Far:

$S \rightarrow A\bar{f} \mid \bar{b}$

$A \rightarrow \bar{b}dA' \mid B\bar{e}A'$

$A' \rightarrow \bar{c}A' \mid \bar{f}dA' \mid \epsilon$

$B \rightarrow \bar{b}dA'g \mid B\bar{e}A'g \mid \bar{b}dA'fh \mid B\bar{e}A'fh \mid \bar{b}h \mid \bar{k}$



LEFT RECURSION IN MORE THAN ONE STEP: ELIMINATION

The Original Grammar:

$S \rightarrow A\bar{f} \mid \bar{b}$
 $A \rightarrow A\bar{c} \mid S\bar{d} \mid B\bar{e}$
 $B \rightarrow A\bar{g} \mid S\bar{h} \mid \bar{k}$

So Far:

$S \rightarrow A\bar{f} \mid \bar{b}$
 $A \rightarrow \bar{b}dA' \mid BeA'$
 $A' \rightarrow \bar{c}A' \mid \bar{f}dA' \mid \epsilon$
 $B \rightarrow \bar{b}dA'g \mid BeA'g \mid \bar{b}dA'fh \mid BeA'fh \mid \bar{b}h \mid \bar{k}$

Finally, eliminate any immediate
Left recursion involving "B"

Next Form

$S \rightarrow A\bar{f} \mid \bar{b}$
 $A \rightarrow \bar{b}dA' \mid BeA'$
 $A' \rightarrow \bar{c}A' \mid \bar{f}dA' \mid \epsilon$
 $B \rightarrow \bar{b}dA'gB' \mid \bar{b}dA'fhB' \mid \bar{b}hB' \mid \bar{k}B'$
 $B' \rightarrow \bar{e}A'gB' \mid \bar{e}A'fhB' \mid \epsilon$

LEFT RECURSION IN MORE THAN ONE STEP: ELIMINATION

The Original Grammar:

$S \rightarrow A\underline{f} \mid \underline{b}$

$A \rightarrow A\underline{c} \mid S\underline{d} \mid B\underline{e} \mid C$

$B \rightarrow A\underline{g} \mid S\underline{h} \mid \underline{k}$

$C \rightarrow B\underline{k}mA \mid AS \mid \underline{j}$

If there is another nonterminal,
then do it next.

So Far:

$S \rightarrow A\underline{f} \mid \underline{b}$

$A \rightarrow \underline{b}dA' \mid B\underline{e}A' \mid CA'$

$A' \rightarrow \underline{c}A' \mid \underline{f}dA' \mid \epsilon$

$B \rightarrow \underline{b}dA'\underline{g}B' \mid \underline{b}dA'\underline{f}hB' \mid \underline{b}hB' \mid \underline{k}B' \mid CA'\underline{g}B' \mid CA'\underline{f}hB'$

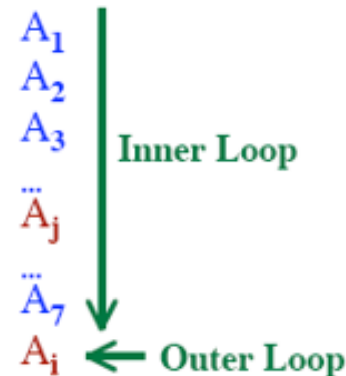
$B' \rightarrow \underline{e}A'\underline{g}B' \mid \underline{e}A'\underline{f}hB' \mid \epsilon$

ALGORITHM FOR ELIMINATING LEFT RECURSION

Assume the nonterminals are ordered $A_1, A_2, A_3, \dots$

(In the example: S, A, B)

```
for each nonterminal  $A_i$  (for  $i = 1$  to  $N$ ) do  
  for each nonterminal  $A_j$  (for  $j = 1$  to  $i-1$ ) do  
    Let  $A_j \rightarrow \beta_1 \mid \beta_2 \mid \beta_3 \mid \dots \mid \beta_N$  be all the rules for  $A_j$   
    if there is a rule of the form  
       $A_i \rightarrow A_j \alpha$   
    then replace it by  
       $A_i \rightarrow \beta_1 \alpha \mid \beta_2 \alpha \mid \beta_3 \alpha \mid \dots \mid \beta_N \alpha$   
    endIf  
  endFor  
  Eliminate immediate left recursion  
    among the  $A_i$  rules  
endFor
```



Left Factoring: Common Prefix Problem

Problem : Uncertain which of 2 rules to choose:

$stmt \rightarrow \text{if } expr \text{ then } stmt \text{ else } stmt$
 $\quad | \text{if } expr \text{ then } stmt$

When do you know which one is valid ?

What's the general form of $stmt$?

$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ $\alpha : \text{if } expr \text{ then } stmt$
 $\beta_1 : \text{else } stmt \quad \beta_2 : \epsilon$

Transform to:

$A \rightarrow \alpha A'$

$A' \rightarrow \beta_1 \mid \beta_2$

EXAMPLE:

$stmt \rightarrow \text{if } expr \text{ then } stmt \text{ rest}$

$rest \rightarrow \text{else } stmt \mid \epsilon$

Left Factoring : Example

$A \rightarrow \underline{ab}B \mid \underline{a}B \mid \underline{cd}g \mid \underline{cde}B \mid \underline{cdf}B$

$\Downarrow$

$A \rightarrow \underline{a}A' \mid \underline{cd}g \mid \underline{cde}B \mid \underline{cdf}B$

$A' \rightarrow \underline{b}B \mid B$

$\Downarrow$

$A \rightarrow \underline{a}A' \mid \underline{cd}A''$

$A' \rightarrow \underline{b}B \mid B$

$A'' \rightarrow g \mid \underline{e}B \mid \underline{f}B$

Left Factoring : Example

$A \rightarrow ad \mid a \mid ab \mid abc \mid b$

$\Downarrow$

$A \rightarrow aA' \mid b$

$A' \rightarrow d \mid \varepsilon \mid b \mid bc$

$\Downarrow$

$A \rightarrow aA' \mid b$

$A' \rightarrow d \mid \varepsilon \mid bA''$

$A'' \rightarrow \varepsilon \mid c$

Assignment

- ▶ Give a CFG for the language of all non-palindromes over $\{0, 1\}^*$.

- ▶ Using the grammar

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid \text{id}$$

Find a leftmost derivation of $\text{id} + \text{id} * \text{id}$.

Find a leftmost derivation of $\text{id} * \text{id} + \text{id}$.

Draw the parse trees.